SEMAINE 5

Modèles auto-régressifs

Règle de la chaîne · NADE / MADE · PixelRNN / PixelCNN

MISE EN CONTEXTE — UNE DISTRIBUTION CONSTRUITE PAS À PAS

En S3, nous avons introduit la factorisation

$$p_\theta(x) = \prod_i p_\theta(x_i \mid x_{<i}).$$

Cette semaine, nous transformons cette identité probabiliste en une véritable famille de modèles : les ***modèles auto-régressifs****. Leur idée est simple : prédire chaque composante à partir de celles qui la précèdent, puis répéter ce mécanisme jusqu'à obtenir une observation complète.*

QUESTION CENTRALE DE LA SEMAINE

Comment décomposer et modéliser une distribution complexe $p(x)$ composante par composante ?

S5 — OBJECTIFS (CE QUE VOUS SAUREZ FAIRE À LA FIN)

1. Lire et expliquer la factorisation auto-régressive de $p_\theta(x)$.
2. Comprendre pourquoi un **ordre** des variables est nécessaire.
3. Expliquer comment FVSBN, NADE et MADE paramètrent les conditionnelles.
4. Comprendre le principe de **PixelRNN** et **PixelCNN**.
5. Distinguer **entraînement parallélisable** et **génération séquentielle**.
6. Expliquer pourquoi la vraisemblance est exacte, mais la génération peut être lente.

RAPPEL EXPRESS — CE QUI VIENT DE S3

Pour une observation $x = (x_1, \ldots, x_n)$, la règle de la chaîne donne :

$$\boxed{p_\theta(x) = \prod_{i=1}^{n} p_\theta(x_i \mid x_{<i}), \qquad x_{<i} = (x_1, \ldots, x_{i-1})}$$

En S3, cette formule servait à montrer comment structurer une distribution complexe. En S5, nous allons voir **comment un réseau apprend concrètement chacune de ces conditionnelles**.

1 Notations indispensables

LIRE LES SYMBOLES SANS AMBIGUÏTÉ

— $x = (x_1, \ldots, x_n)$: une observation complète (image, texte, audio, etc.).
— x_i : la i-ème composante de cette observation.
— $x_{<i} = (x_1, \ldots, x_{i-1})$: les composantes placées **avant** x_i dans l'ordre choisi.
— $p_\theta(x_i \mid x_{<i})$: distribution prédite pour x_i connaissant le passé.
— θ : paramètres du réseau qui calcule ces distributions conditionnelles.
— $\hat{x}_i$: paramètre prédit par le réseau ; dans le cas binaire, il représente souvent $p_\theta(x_i = 1 \mid x_{<i})$.
— $\sigma(\cdot)$: fonction sigmoïde ; elle transforme un score réel en une valeur dans $[0, 1]$.
— softmax$(\cdot)$: transforme plusieurs scores en probabilités positives dont la somme vaut 1.

ATTENTION AUX INDICES

En S3, $x^{(i)}$ désignait souvent la i-ème **observation du jeu de données**. Ici, x_i désigne la i-ème **composante d'une même observation**. Ces deux notations ne jouent pas le même rôle.

2 Règle de la chaîne : cœur des modèles auto-régressifs

2.1 Décomposer une loi jointe

FACTORISATION AUTO-RÉGRESSIVE

Pour comprendre la formule suivante, pensez à une observation complète $x = (x_1, \dots, x_n)$ que l'on construit **une composante après l'autre**. À chaque étape, on décrit la prochaine composante en utilisant uniquement celles qui la précèdent :

$$p(x_1, \dots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_{<i}).$$

Cette identité est **toujours vraie** : elle vient de la règle de la chaîne des probabilités.

DE L'IDENTITÉ AU MODÈLE APPRIS

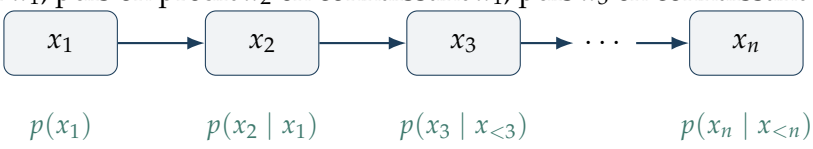
La règle de la chaîne est une identité sur toute distribution $p(x)$. Dans un modèle auto-régressif, nous paramétrons chacune de ses conditionnelles avec un réseau de paramètres θ :

$$p_\theta(x) = \prod_{i=1}^{n} p_\theta(x_i \mid x_{<i}).$$

Lecture pas à pas :

$$p(x_1, x_2, x_3) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_1, x_2).$$

On commence par x_1, puis on prédit x_2 en connaissant x_1, puis x_3 en connaissant ce qui précède.



2.2 Pourquoi faut-il choisir un ordre ?

Un modèle auto-régressif doit décider ce qui signifie « le passé ». Pour du texte, l'ordre naturel est généralement gauche → droite. Pour une image, on peut choisir un parcours *raster* : ligne par ligne, du coin supérieur gauche vers le coin inférieur droit.

LA RÈGLE DE LA CHAÎNE EST VRAIE ; L'ORDRE RESTE UN CHOIX DE MODÉLISATION

Toute permutation des variables donne une factorisation mathématiquement valide. En revanche, changer l'ordre change les conditionnelles que le réseau doit apprendre et peut donc modifier ses performances et ses biais inductifs.

EXEMPLE — UNE IMAGE BINAIRE 2×2

Après linéarisation en (x_1, x_2, x_3, x_4) :

$$p(x_1, x_2, x_3, x_4) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2)p(x_4 \mid x_1, x_2, x_3).$$

Lecture : pour générer l'image, on tire d'abord x_1, puis x_2, puis x_3, puis x_4, toujours en conditionnant sur ce qui a déjà été généré.

3 Paramétrer une conditionnelle

3.1 Cas binaire : une Bernoulli

Si $x_i \in \{0, 1\}$, le réseau doit prédire la probabilité que la prochaine valeur soit 1. Il produit d'abord un score réel s_i, puis la fonction sigmoïde transforme ce score en une valeur comprise entre 0 et 1, interprétable comme une probabilité :

$$\hat{x}_i = p_\theta(x_i = 1 \mid x_{<i}) = \sigma(s_i), \qquad \sigma(s) = \frac{1}{1 + e^{-s}}.$$

Une fois $\hat{x}_i$ obtenue, il faut attribuer une probabilité à la valeur réellement observée x_i. La même écriture suivante couvre automatiquement les deux cas $x_i = 1$ et $x_i = 0$:

$$p_\theta(x_i \mid x_{<i}) = \hat{x}_i^{x_i}(1 - \hat{x}_i)^{1-x_i}.$$

COMMENT LIRE CETTE FORMULE ?

— si $x_i = 1$, le facteur vaut $\hat{x}_i$;

— si $x_i = 0$, le facteur vaut $1 - \hat{x}_i$;

— le réseau est donc récompensé lorsqu'il attribue une forte probabilité à la valeur réellement observée.

LIEN AVEC LES PERTES CONNUES

Pour une variable binaire, la NLL de la Bernoulli est exactement l'entropie croisée binaire :

$$-\log p_\theta(x_i \mid x_{<i}) = -\Big[x_i \log \hat{x}_i + (1 - x_i)\log(1 - \hat{x}_i)\Big].$$

Pour une variable à K catégories, la NLL correspond à l'entropie croisée multiclasses. **Idée :** une perte classique de classification devient ici une perte de génération, car le modèle prédit la prochaine composante plutôt qu'une étiquette global.

3.2 Cas multi-classe : softmax

Si x_i peut prendre K valeurs, une seule probabilité ne suffit plus. Le réseau produit alors un vecteur de K scores a_i, un score par valeur possible. Le softmax transforme ces scores en K probabilités positives dont la somme vaut 1 :

$$(p_i^1, \ldots, p_i^K) = \mathrm{softmax}(a_i), \qquad \sum_{k=1}^{K} p_i^k = 1.$$

Pour un pixel 8 bits, on peut par exemple avoir $K = 256$ valeurs possibles.

4 Apprentissage par MLE / NLL

VRAISEMBLANCE EXACTE

La factorisation transforme la probabilité jointe en un produit de conditionnelles. En prenant le logarithme, ce produit devient une somme : chaque position apporte donc sa propre contribution à la log-vraisemblance totale. Parce que toutes les conditionnelles sont explicitement calculables :

$$\log p_\theta(x) = \sum_{i=1}^{n} \log p_\theta(x_i \mid x_{<i}).$$

On peut donc appliquer directement le MLE de S3, ou de manière équivalente minimiser la NLL.

NLL SUR UN JEU DE DONNÉES

Pour un jeu de données $\mathcal{D} = \{x^{(1)}, \ldots, x^{(M)}\}$, la NLL totale additionne les positions de toutes les observations :

$$\mathcal{L}_{\text{NLL}}(\theta) = -\sum_{m=1}^{M} \sum_{i=1}^{n} \log p_\theta\left(x_i^{(m)} \mid x_{<i}^{(m)}\right).$$

Lecture : le modèle est pénalisé chaque fois qu'il attribue une faible probabilité à une composante réellement observée.

DU CONCEPT AU CODE — NLL AUTO-RÉGRESSIVE

Le pseudo-code ci-dessous ne décrit pas une implémentation optimisée ; il traduit simplement la formule mathématique étape par étape. À chaque position i, on évalue la probabilité attribuée à la vraie valeur x_i, puis on additionne son logarithme.

```
log_p = 0.0
for i in range(n):
    p_i = model.prob(x[i], context=x[:i])
    log_p += log(p_i)

nll = -log_p
```

À observer : chaque position ajoute un terme à la log-vraisemblance. En pratique, les architectures modernes calculent souvent plusieurs conditionnelles en parallèle pendant l'entraînement.

ENTRAÎNEMENT PARALLÈLE NE SIGNIFIE PAS GÉNÉRATION PARALLÈLE

Pendant l'entraînement, l'observation complète x est connue. Une architecture masquée peut donc calculer plusieurs distributions conditionnelles en une seule passe sans révéler le futur. Pendant la génération, en revanche, x_{i+1} n'existe pas encore tant que x_i n'a pas été échantillonné : la génération reste séquentielle.

5 Repères historiques : FVSBN, NADE et MADE

Cette section sert à comprendre comment on est passé de conditionnelles simples à des réseaux auto-régressifs plus efficaces. L'objectif n'est pas de mémoriser chaque détail d'architecture, mais de voir l'évolution de l'idée.

5.1 FVSBN : une logistique par variable

FVSBN — IDÉE

FVSBN est le cas le plus simple : pour prédire x_i, on forme d'abord une combinaison linéaire des variables précédentes $x_1, \ldots, x_{i-1}$. La sigmoïde transforme ensuite cette combinaison en probabilité :

$$\hat{x}_i = p(x_i = 1 \mid x_{<i}) = \sigma\left(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i x_j\right).$$

Lecture : α_j^i mesure l'influence de x_j sur la prédiction de x_i. Le modèle est simple et interprétable, mais le nombre de paramètres croît quadratiquement avec n.

COÛT EN PARAMÈTRES

Sans partage de paramètres, le modèle comporte

$$\sum_{i=1}^{n}(i-1) = \frac{n(n-1)}{2}$$

coefficients de dépendance, auxquels s'ajoutent les biais. Le nombre de paramètres croît donc quadratiquement avec n.

5.2 NADE : partager une représentation cachée

NADE conserve le contexte $x_{<i}$, mais le transforme d'abord en une **représentation cachée partagée** h_i. Cette représentation résume le passé avant que le modèle ne calcule la distribution conditionnelle de x_i :

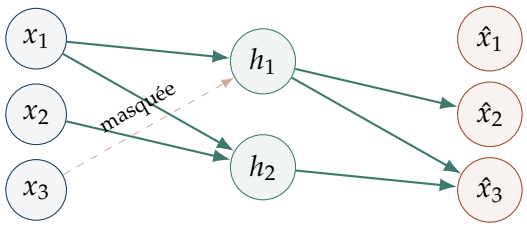
$$h_i = \sigma(W_{:,<i} x_{<i} + c), \qquad \hat{x}_i = \sigma(\alpha_i^\top h_i + b_i).$$

LECTURE EN DEUX ÉTAPES

1. h_i résume le contexte $x_{<i}$.
2. $\hat{x}_i$ transforme ce résumé en probabilité pour x_i.
Le partage de poids permet d'éviter de construire un réseau indépendant pour chaque position.

5.3 MADE : masquer les connexions

MADE applique des masques binaires aux matrices de poids d'un MLP. Ces masques garantissent qu'il n'existe aucun chemin de calcul permettant à la sortie $\hat{x}_i$ d'utiliser x_i ou une variable future x_j avec $j \geq i$.



Les masques suppriment toute connexion ou tout chemin qui permettrait à $\hat{x}_i$ d'utiliser x_i ou le futur.

CONSÉQUENCE DE MADE

— toutes les conditionnelles peuvent être calculées en une seule passe à l'entraînement ;
— la vraisemblance reste exacte ;
— la génération reste séquentielle.

NE PAS CONFONDRE AVEC UN AUTOENCODEUR CLASSIQUE

Un autoencodeur classique apprend à reconstruire x à partir d'une représentation interne. MADE utilise une architecture ressemblant à un MLP encodeur/décodeur, mais ses masques imposent les dépendances auto-régressives nécessaires pour définir

$$p(x) = \prod_i p(x_i \mid x_{<i}).$$

Nous étudierons les autoencodeurs comme famille à part entière en S6.

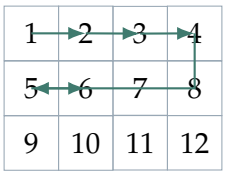
6 PixelRNN : appliquer l'auto-régressif aux images

6.1 Ordre raster et dépendances longues

PixelRNN applique exactement la même règle de chaîne aux images. On transforme conceptuellement l'image en une séquence ordonnée de pixels ; à l'étape t, le pixel courant est prédit à partir de tous les pixels qui le précèdent dans cet ordre :

$$p(x) = \prod_{t=1}^{n} p(x_t \mid x_{1:t-1}).$$

Un RNN adapté aux images résume les pixels précédents avant de prédire la distribution du pixel courant.



Ordre raster : gauche → droite, puis ligne suivante.

Pour une image couleur, on peut aussi imposer un ordre à l'intérieur d'un pixel : rouge, puis vert sachant rouge, puis bleu sachant rouge et vert.

PIXELRNN — À RETENIR

— les conditionnelles peuvent capturer des dépendances longues grâce au mécanisme récurrent ;
— l'entraînement est coûteux car le calcul est fortement séquentiel ;
— la génération est elle aussi séquentielle, pixel par pixel.

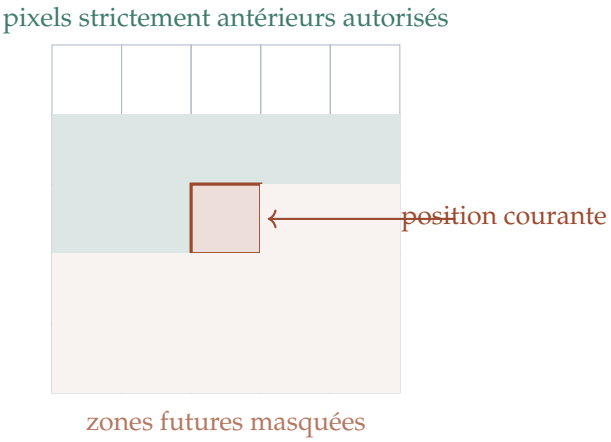
7 PixelCNN : convolutions masquées

7.1 Pourquoi une convolution standard ne suffit pas ?

Une convolution classique utilise le voisinage autour d'un pixel, y compris parfois des pixels placés **après** la position courante dans l'ordre raster. Cela violerait la factorisation auto-régressive.

PIXELCNN *(problème → échec → solution)*

Problème : une convolution standard peut « voir le futur ».
Échec : le réseau pourrait utiliser x_i ou des pixels futurs pour prédire x_i.
Solution : masquer certains coefficients du filtre afin que chaque sortie ne dépende que de $x_{<i}$.



NUANCE ARCHITECTURALE (À CONNAÎTRE SANS MÉMORISER)

Dans PixelCNN, la première couche utilise typiquement un **masque A**, qui exclut la valeur du pixel courant. Les couches suivantes peuvent employer un **masque B**, qui conserve l'ordre auto-régressif tout en autorisant certaines dépendances internes déjà construites par le réseau.

7.2 Entraînement parallèle, génération séquentielle

LA DISTINCTION ESSENTIELLE

À l'entraînement : l'image complète est disponible et le masque empêche les fuites d'information ; les conditionnelles peuvent donc être calculées en parallèle sur GPU.

À la génération : le prochain pixel n'est pas encore connu ; il faut l'échantillonner avant de passer au suivant.

DU CONCEPT AU CODE — GÉNÉRATION AUTO-RÉGRESSIVE

Pendant la génération, on ne connaît pas encore la suite complète : il faut produire une valeur, l'ajouter au contexte, puis recommencer. Le pseudo-code suivant matérialise cette dépendance séquentielle :

```
x = empty_sequence(n)

for i in range(n):
    probs_i = model.predict_next(x[:i]) # p(x_i | x_<i)
    x[i] = sample(probs_i)
```

À observer : même si le réseau calcule rapidement les probabilités, la boucle de génération reste dépendante de l'ordre : x_i doit exister avant de générer x_{i+1}.

8 Comparer les principales familles auto-régressives

MÊME LOI, MÉCANISMES DIFFÉRENTS

PixelRNN et PixelCNN modélisent tous deux

$$p_\theta(x) = \prod_i p_\theta(x_i \mid x_{<i}).$$

La différence ne porte pas sur la loi probabiliste, mais sur l'architecture utilisée pour calculer les conditionnelles : mémoire récurrente pour PixelRNN, convolutions masquées pour PixelCNN.

Modèle	Idée / architecture	Entraînement	Génération
FVSBN	Régressions logistiques	simple, mais beaucoup de paramètres	séquentielle
NADE	MLP + poids partagés	plus efficace que FVSBN	séquentielle
MADE	MLP avec masques	conditionnelles en une passe	séquentielle
PixelRNN	RNN pour images	fortement séquentiel	séquentielle, lente
PixelCNN	CNN masqué	largement parallélisable	séquentielle, pixel par pixel

AVANTAGES

— **Vraisemblance exacte et calculable** : $\log p_\theta(x) = \sum_i \log p_\theta(x_i \mid x_{<i})$.
— **Apprentissage direct par MLE/NLL**.
— **Échantillonnage conceptuellement simple** : prédire puis tirer la prochaine composante.
— Architecture flexible : MLP, RNN, CNN masqué ou transformeur causal.

LIMITES

— il faut choisir un ordre de factorisation ;
— la génération est intrinsèquement séquentielle ;
— le coût devient important pour les longues séquences, l'audio ou les images haute résolution ;
— une bonne vraisemblance ne signifie pas automatiquement que tous les échantillons seront perceptuellement satisfaisants.

9 Synthèse de la semaine 5

CE QU'IL FAUT RETENIR

— Un modèle auto-régressif construit une distribution jointe à partir de conditionnelles. Autrement dit, la probabilité d'une observation complète est obtenue en combinant les probabilités de chaque composante sachant celles qui la précèdent :

$$p_\theta(x) = \prod_i p_\theta(x_i \mid x_{<i}).$$

— La vraisemblance est **exacte** lorsque chaque conditionnelle est calculable.
— L'entraînement peut parfois être parallélisé grâce aux masques.
— La génération reste **séquentielle** : $x_1 \to x_2 \to \cdots \to x_n$.
— MADE et PixelCNN imposent structurellement l'interdiction de « regarder le futur ».

AUTO-VÉRIFICATION

Pouvez-vous expliquer, sans formule supplémentaire :

1. pourquoi la règle de la chaîne permet une vraisemblance exacte ;
2. pourquoi PixelCNN peut être entraîné en parallèle mais doit générer séquentiellement ;
3. à quoi sert un masque auto-régressif ;
4. quelle différence conceptuelle existe entre un autoencodeur classique et MADE ?

TRANSITION VERS LA SEMAINE 6

Les modèles auto-régressifs savent construire directement $p_\theta(x)$, mais ils paient cette exactitude par une génération pas à pas. En S6, nous changerons de stratégie : au lieu de factoriser explicitement toutes les composantes de x, nous apprendrons à ***compresser*** *une observation dans une représentation puis à la* ***reconstruire****. Ce sera le rôle des* ***autoencodeurs profonds****.*